

AI ACADEMY, NATIONAL AI CENTER

MLE-AI-201 – MACHINE LEARNING – AI ENGINEERING (MODULE 2) – COHORT II

Homework 1 (Weeks 1–2)

KNN & Metrics, Linear & Logistic Regression from Scratch

Released: September 20, 2026**Due:** **October 4, 2026 at 23:59 (UTC+4)****Points:** 100 + 10 bonus**Est. time:** 8–10 hours**Version:** v1.0**Submission:** Single .zip on Moodle

At a glance

Prerequisites. Lectures from Week 1 (INTRO & ML LANDSCAPE) and Week 2 (SUPERVISED LEARNING I). Comfort with NumPy, vectors, norms, gradients, and basic probability.

What you will build. Four core algorithms in NumPy *from scratch*, each trained on a small built-in `scikit-learn` dataset and compared against the library implementation:

1. k -Nearest Neighbors (classification).
2. The classification metrics: accuracy, precision, recall, F_1 .
3. Linear regression (gradient descent).
4. Logistic regression (gradient descent).

Deliverables. A single .zip containing (i) a PDF report built from `report_template.tex` (≤ 6 pages), (ii) your completed code in `src/`, (iii) a signed `pledge.txt`.

1. Logistics

1.1 This is HW1 (Weeks 1–2)

Per the syllabus, each homework covers two weeks and requires **an algorithm implemented from scratch and its library equivalent, with the results compared and any discrepancy explained**. HW1 carries **10 marks: 8 for this submission and 2 for a short live check** in the lab session after the deadline, where you talk through your own code. The bonus below can raise the submission mark but the total is capped at 8.

1.2 Collaboration policy

You may discuss concepts at a high level with classmates, but the work you submit must be your own: no sharing of code, plots, or written analysis, even partial. Acknowledge any collaborator by name at the top of your report.

1.3 AI / LLM tool policy

AI assistants may be used for homework. **Any use must be declared in your submission:** name the tool, what you used it for, which parts it affected, and how you checked the output. You must be able to explain and reproduce your work without AI — the live check is closed-book. Submitting code you cannot explain results in **zero credit on the affected component**.

1.4 Late policy

Late work is *capped, not deducted*: up to and including 24h late is capped at 80%; more than 24h and up to 48h is capped at 60%; more than 48h scores 0 unless documented and pre-approved. The bands do not overlap. Extensions must be requested *before* the deadline via `mle-ai-201-staff@aiacademy.edu.az`.

1.5 Reproducibility

Seed every random operation with `np.random.seed(42)`. We spot-check by re-running submissions; results that do not reproduce lose up to 5 points.

2. Background

2.1 k -Nearest Neighbors

KNN is a non-parametric classifier: there is no training beyond memorizing the data. For a query $\mathbf{x} \in \mathbb{R}^p$ it finds the k nearest training points by Euclidean distance and returns the majority label:

$$\hat{y}(\mathbf{x}) = \arg \max_c \sum_{i \in \mathcal{N}_k(\mathbf{x})} \mathbf{1}\{y^{(i)} = c\}, \quad d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_j (x_j - y_j)^2}.$$

2.2 Classification metrics

With TP, FP, TN, FN counts,

$$\text{Acc} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}, \quad \text{Prec} = \frac{\text{TP}}{\text{TP} + \text{FP}},$$

$$\text{Rec} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{F}_1 = \frac{2 \cdot \text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}}.$$

2.3 Linear regression

Model $\hat{y} = \mathbf{X}\mathbf{w} + b$. Minimize mean squared error $J(\mathbf{w}, b) = \frac{1}{N} \|\mathbf{X}\mathbf{w} + b - \mathbf{y}\|_2^2$ by gradient descent:

$$\nabla_{\mathbf{w}} J = \frac{2}{N} \mathbf{X}^\top (\mathbf{X}\mathbf{w} + b - \mathbf{y}), \quad \partial_b J = \frac{2}{N} \sum_i (\hat{y}_i - y_i).$$

2.4 Logistic regression

Model $\hat{p} = \sigma(\mathbf{X}\mathbf{w} + b)$ with $\sigma(z) = 1/(1 + e^{-z})$. Minimize binary cross-entropy $J = -\frac{1}{N} \sum_i [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$; the gradient has the same clean form:

$$\nabla_{\mathbf{w}} J = \frac{1}{N} \mathbf{X}^\top (\hat{\mathbf{p}} - \mathbf{y}), \quad \partial_b J = \frac{1}{N} \sum_i (\hat{p}_i - y_i).$$

3. Setup

3.1 Environment

Python 3.11 with `numpy`, `matplotlib`, and `scikit-learn` (used for datasets and sanity checks only), plus `pytest`. A `requirements.txt` is included.

3.2 Datasets (all built into scikit-learn — no downloads)

- **Classification** (Parts 1 & 2.2): `sklearn.datasets.load_breast_cancer` — a binary target, 30 numeric features.
- **Regression** (Part 2.1): `sklearn.datasets.load_diabetes` — a continuous target, 10 numeric features.

Always split with `train_test_split(..., random_state=42)` and standardize features (fit the scaler on the training split only).

3.3 Directory layout (provided)

```
Machine_learning_HW_WEEK_1_2/
|-- statement.pdf           # this file
|-- report_template.tex     # fill this in, compile to report.pdf
|-- report.pdf              # <-- you add this
|-- requirements.txt
|-- README.md
|-- pledge.txt              # sign it
|-- src/
|   |-- knn.py               # Part 1
|   |-- metrics.py           # Part 1
|   |-- linear_regression.py # Part 2.1 (+ bonus)
|   |-- logistic_regression.py # Part 2.2 (+ bonus)
|-- tests/
|   |-- test_knn.py
|   |-- test_metrics.py
|-- figures/                 # any plots referenced in the report
```

4. Problems

4.1 Part 1 — KNN & Metrics [50 pts]

Use the **breast cancer** dataset throughout Part 1. Fill in the stubs in `src/knn.py` and `src/metrics.py`. **Do not use sklearn except for loading data and the explicit sanity checks.**

- 1.1 KNN from scratch. [20 pts]** Complete the KNN class (`fit`, `predict`) using vectorized NumPy for the Euclidean distance. **No Python for loop over training points.** Support an integer hyperparameter k .
- 1.2 Metrics from scratch. [20 pts]** In `src/metrics.py` implement `accuracy`, `precision`, `recall`, and `f1_score` from the confusion-matrix counts. Handle the zero-denominator edge case (return 0.0).
- 1.3 Train, evaluate, compare. [10 pts]** Train your KNN for $k \in \{1, 3, 5, 7, 9, 11, 15, 21\}$, evaluate all four metrics on the test set, and report the best k by F_1 . Confirm your predictions match `sklearn.neighbors.KNeighborsClassifier` with the same k , and that your metrics match `sklearn.metrics` within 10^{-9} .

Part 1 rubric. A Python loop over training points in 1.1 loses 4 pts. Metrics that disagree with `sklearn.metrics` lose 4 pts. A plot or table of the metric-vs- k sweep is required for 1.3.

4.2 Part 2 — Linear & Logistic Regression [50 pts]

Fill in the stubs in `src/linear_regression.py` and `src/logistic_regression.py`. Both must train by **gradient descent** (no normal equations, no `sklearn` for the core fit).

2.1 Linear regression from scratch. [25 pts] On the **diabetes** dataset, implement `LinearRegression` with `fit` (batch gradient descent) and `predict`. Train it, plot the training-loss curve (MSE vs. epoch), and report test-set MSE and R^2 . Compare against `sklearn.linear_model.LinearRegression` and explain any gap.

2.2 Logistic regression from scratch. [25 pts] On the **breast cancer** dataset, implement `LogisticRegression` with `fit` (gradient descent on cross-entropy), `predict_proba`, and `predict`. Train it, plot the loss curve, and report test-set accuracy, precision, recall, and F_1 (reuse your Part 1 metrics). Compare against `sklearn.linear_model.LogisticRegression` and explain any gap.

Part 2 rubric. Using the normal equation or `sklearn` for the core fit loses 10 pts. Both loss curves must show a monotonically decreasing trend; a diverging loss usually means the learning rate is too high or features are unscaled.

Bonus — Regularization [+10 bonus]

Add regularization to *either* regressor (a `penalty/lambda` argument in its `fit`). Report how the learned weight norm and test performance change as `lambda` varies.

- **[+5 bonus] L1 (Lasso).** Add $\lambda \|\mathbf{w}\|_1$ to the loss; the (sub)gradient adds $\lambda \text{sign}(\mathbf{w})$. Show that large λ drives some weights to (near) zero.
- **[+5 bonus] L2 (Ridge).** Add $\lambda \|\mathbf{w}\|_2^2$ to the loss; the gradient adds $2\lambda \mathbf{w}$. Show that it shrinks weights smoothly toward zero.

Do *not* regularize the bias term. State clearly in the report which bonus(es) you attempted.

5. Submission

5.1 Checklist

- ☐ `report.pdf` (≤ 6 pages) built from `report_template.tex`, with your name, ID, and date.
- ☐ `src/` contains your completed `knn.py`, `metrics.py`, `linear_regression.py`, `logistic_regression.py`.
- ☐ `pytest` runs cleanly on the provided tests.
- ☐ Every random call is seeded; results reproduce.
- ☐ `pledge.txt` is signed and dated; AI use is declared.

5.2 Format

A single `.zip` named `lastname_firstname_hw1.zip` uploaded to Moodle.

6. Honor Pledge

Copy the following into `pledge.txt` and sign with your name and the date:

I, [Name], affirm that the work in this submission is my own. I have not shared or received code, plots, or written analysis for this assignment. Where I used AI assistants, I did so only as permitted by the AI/LLM policy in §1, and I have declared that use. I can explain every line of code and every paragraph of writing in my submission.

Signed: _____ Date: _____

References

- [1] James, Witten, Hastie, Tibshirani & Taylor. *An Introduction to Statistical Learning with Applications in Python* — Ch. 2–4. statlearning.com.
- [2] Hastie, Tibshirani & Friedman. *The Elements of Statistical Learning*, 2nd ed. — Ch. 3 (linear methods), Ch. 13 (KNN).
- [3] scikit-learn developers. *User Guide*. scikit-learn.org.

End of Homework 1. Good luck — implement it yourself, then let the library check your work.